

SymPy Bug Report

1 Bug 2: solveset Returns All Complexes for $\text{sqrt}(x^2) = x$

1.1 Minimal Reproducer

```
from sympy import Eq, N, S, simplify, solveset, sqrt, symbols

x = symbols("x")
eq = Eq(sqrt(x**2), x)
sol = solveset(eq, x, domain=S.Complexes)

print("solveset result:", sol)
print("residual at x = -1:", simplify(eq.lhs.subs(x, -1) - eq.rhs.subs(x, -1)))
print("numeric residual:", N(eq.lhs.subs(x, -1) - eq.rhs.subs(x, -1), 50))
```

1.2 Observed Behavior

```
solveset result: Complexes  
residual at x = -1: 2  
numeric residual: 2.0000000000000000000000000000000000000000000000000000000
```

SymPy reports that every complex number solves

$$\sqrt{x^2} = x.$$

This is incorrect because the returned set includes $x = -1$, and direct substitution gives

$$\sqrt{(-1)^2} - (-1) = 1 + 1 = 2.$$

1.3 Expected Behavior

The result must be a proper subset of $\mathbb{C}$ and must exclude $x = -1$. With the principal square root, the complex solutions are the branch-restricted set

$$\{z \in \mathbb{C} : \operatorname{Re}(z) > 0\} \cup \{it : t \in \mathbb{R}, t \geq 0\}.$$

In particular, over the real line the solution set is $[0, \infty)$, not all real or complex values.

1.4 Mathematical Explanation

The identity $\sqrt{x^2} = x$ is not valid for arbitrary complex x . SymPy's $\sqrt{\cdot}$ denotes the principal square root, so clearing the square root by squaring both sides can introduce extraneous solutions unless the principal-branch condition is preserved.

For real x , the familiar identity is

$$\sqrt{x^2} = |x|,$$

so the equation $\sqrt{x^2} = x$ holds exactly for $x \geq 0$. The same issue persists over the complex numbers: the principal square root chooses a branch, and $\sqrt{z^2}$ agrees with z only on the corresponding branch-restricted subset. The returned answer $\mathbb{C}$ is therefore not a harmless alternative representation or a missing simplification; it contains concrete values that do not satisfy the original equation.

1.5 Source-Level Diagnosis

The diagnosis identifies the immediate source-level cause in `sympy/solvers/solveset.py`, specifically in `_solve_radical` around line 1094. The traced call path is:

`solveset` $\rightarrow$ `_solveset` $\rightarrow$ `unrad` $\rightarrow$ `_solve_radical`.

For $\sqrt{x^2} - x = 0$, direct inversion does not reduce the equation, so `_solveset` calls `unrad`. The radical-removal step returns the unradicalized equation $0 = 0$ with no covering variables. In other words, squaring has turned the original branch-sensitive equation into a tautology.

The problematic behavior is that `_solve_radical` treats the solution set of this unradicalized equation as acceptable even when it is non-finite. The diagnosis notes that `unrad` produces a superset equation rather than an equivalent equation, but the checker in `_solve_radical` validates only certain finite or set-combination cases. Non-finite set results such as $\mathbb{C}$ can therefore be returned unchanged, leaving no check against the original equation.

```
if not cov:
    result = solveset_solver(eq, symbol) - ...
...
else:
    return solutions
```

This behavior causes the observed failure because the branch condition lost during radical clearing is never restored. A plausible fix direction identified by the diagnosis is for `_solve_radical` not to return non-finite radical-clearing results unconditionally. It should either preserve the original equation as a condition, for example through a `ConditionSet`, or derive and apply the branch conditions introduced by radical removal.

1.6 Additional Failing Instances

The independent verification covers the representative case and shifted variants

$$\sqrt{(x+a)^2} = x+a$$

for $a \in \{0, 1, 2, 3, 4, 5\}$. In each case, SymPy returns $\mathbb{C}$, while the explicit counterexample $x = -a-1$ makes $x+a = -1$. Therefore the residual is again

$$\sqrt{(-1)^2} - (-1) = 2,$$

so the same principal-square-root branch error appears across the family.

Input / Expression	SymPy behavior	Expected behavior
<code>solveset($\sqrt{x^2} = x$, x, $\mathbb{C}$)</code>	Returns $\mathbb{C}$; includes $x = -1$.	A proper subset of $\mathbb{C}$; exclude $x = -1$.
<code>solveset($\sqrt{(x+1)^2} = x+1$, x, $\mathbb{C}$)</code>	Returns $\mathbb{C}$; includes $x = -2$.	A proper subset of $\mathbb{C}$; exclude $x = -2$.
<code>solveset($\sqrt{(x+2)^2} = x+2$, x, $\mathbb{C}$)</code>	Returns $\mathbb{C}$; includes $x = -3$.	A proper subset of $\mathbb{C}$; exclude $x = -3$.
<code>solveset($\sqrt{(x+3)^2} = x+3$, x, $\mathbb{C}$)</code>	Returns $\mathbb{C}$; includes $x = -4$.	A proper subset of $\mathbb{C}$; exclude $x = -4$.
<code>solveset($\sqrt{(x+4)^2} = x+4$, x, $\mathbb{C}$)</code>	Returns $\mathbb{C}$; includes $x = -5$.	A proper subset of $\mathbb{C}$; exclude $x = -5$.
<code>solveset($\sqrt{(x+5)^2} = x+5$, x, $\mathbb{C}$)</code>	Returns $\mathbb{C}$; includes $x = -6$.	A proper subset of $\mathbb{C}$; exclude $x = -6$.

1.7 Related Issues Assessment

The bug-finding harness performed a best-effort deduplication check and classified this as a new instance of a known failure mode. The closest public material is SymPy’s own square-root documentation, which explicitly explains that $\sqrt{x^2}$ does not simplify to x in general and even uses $x = -1$ as a counterexample, together with general `solveset` documentation stating that returned sets should contain solutions of the input equation. Those references establish that the mathematical pitfall is known, but they do not report the specific solver behavior `solveset( $\sqrt{x^2} = x$ ,  $x$ ,  $\mathbb{C}$ ) =  $\mathbb{C}$` . The case remains individually actionable because it is a reproducible solver correctness failure with a concise regression test.

1.8 Proposed SymPy Regression Test

```
import pytest

from sympy import Eq, S, simplify, solveset, sqrt, symbols

@pytest.mark.parametrize("a", [0, 1, 2, 3, 4, 5])
def test_solveset_sqrt_square_not_all_complexes(a):
    x = symbols("x")
    eq = Eq(sqrt((x + a)**2), x + a)
    sol = solveset(eq, x, domain=S.Complexes)
    bad = -a - 1

    assert sol != S.Complexes
    assert simplify(eq.lhs.subs(x, bad) - eq.rhs.subs(x, bad)) != 0
```